

WhatsApp Group Wrapped

Author - Ansh Agrawal

Project Specification

Python · JavaScript · HTML / CSS

1. Overview

This project involves building a browser-based, Spotify Wrapped-style slideshow that visualises social dynamics found in a WhatsApp group chat. The system has three parts: a Python chat generator, a Python statistics parser, and a JavaScript front-end. Python and JavaScript communicate solely through a JSON file that you design.

2. System Architecture

The pipeline flows as follows:

```
vocabulary.txt → chat.py → chat.txt → analyze.py → data.json →  
app.js → Browser
```

Suggested directory structure:

```
whatsapp-wrapped/  
├── generator/chat.py  
├── parser/analyze.py  
├── web/ (index.html, style.css, app.js, *.html, *.css, *.js)  
├── chat.txt  
├── data.json  
└── vocabulary.txt
```

3. Chat Generation (chat.py)

Write a Python script that generates a group chat from a provided vocabulary.txt file (comma-separated words), following this exact format:

```
DD/MM/YY, HH:MM - Sender_Name: Message text
```

The chat must contain clear, distinct personalities. Simple random word selection is insufficient. You must implement social behaviour patterns such as:

- Frequency weights: some users message far more often than others (e.g. a "Chatterbox")
- Time constraints: certain users are only active during specific hours (e.g. a "Night Owl" active 12am–4am)
- Message length: some users send one-liners while others write long paragraphs
- Reply streaks: one user sends several consecutive messages without any response (a "Ghoster" effect)

Use numpy functions where helpful. Each personality pattern must be clearly implemented and commented in the code. There should be at least 7 group members in the WhatsApp chat (not in your project team!). All of the (simulated) WhatsApp group members must have taken CS-108 this semester :-)

Run command:

```
python chat.py <vocabulary.txt>
```

4. Statistics Parser (analyze.py)

Write a Python parser that reads chat.txt, computes statistics, and outputs data.json. You must implement at minimum the following 10 statistics:

#	Statistic	Description
1	total_messages	Message count per person
2	total_words	Word count per person
3	night_owl	Most active sender between 12am – 4am
4	ghost	Person whose messages receive the fewest direct replies
5	conversation_starter	Most frequent sender after a long silence
6	most_used_emoji	Top 3 emojis per person
7	busiest_day	Single day with the most messages
8	longest_silence	Longest gap with zero messages (in hours/days)
9	avg_response_time	Median minutes before someone replies to each person
10	hype_person	Person with the fastest average reply time

In addition, you must define and implement at least one custom statistic that is specific to your group chat - something that a generic tool would not compute. Explain your reasoning for this metric in the report and in inline comments.

Important: several terms are intentionally left undefined. You must decide and document your own definitions for:

- What counts as a "direct reply" to a message?
- What gap in time defines a "long silence" for the conversation_starter metric?
- What makes a day "busy" vs "chaotic"?

State each definition in your code (as a comment) and justify your choice in the report.

Run command:

```
python analyze.py <chat.txt>
```

Allowed Python modules: numpy, datetime, json, re, pathlib, sys, os, time.

Edit - Additionally allowed libraries - random, emoji

If you want to use any other library, confirm from TAs. If that doesn't break the purpose of the assignment, it will get approved.

5. The Wrapped Experience (app.js / index.html)

Build a full-screen, click-to-advance browser slideshow. Data is read only from data.json. All visual design (colour scheme, typography, animations, transitions) is your responsibility.

5.1 Global Slides

Each of the required statistics must have its own slide with an appropriate visualisation (e.g. bar chart, timeline, emoji grid). At minimum, think carefully about which chart type best represents each statistic and why.

5.2 Per-Person Profile Slides

The slideshow must include a dedicated profile for each member of the group chat. Each profile slide should display that person's individual statistics in one place, including:

- Total messages and total words sent
- Their activity heatmap or time-of-day distribution (bar chart or similar)
- Their top 3 most-used emojis
- Their average response time

- Any other relevant per-person metrics your parser computes

The per-person slides should be visually consistent with the rest of the slideshow but clearly distinguishable as a "profile" view. Add a functionality that lets the user choose whose profile they want to view. The user may wish to view multiple profiles at once.

6. data.json Schema

You must design the JSON schema before writing either the parser or the front-end. The schema should cleanly separate:

0

- Group-level statistics (one value per stat, e.g. `busiest_day`, `longest_silence`)
- Per-person statistics (a dictionary keyed by name, containing all per-person metrics)

The schema is a shared contract between `analyze.py` and `app.js`. Both sides must agree on it before implementation begins. Document the schema in your README.

7. Git & Collaboration Requirements

One project member should create a **private** git repository and share with the other team member. You can use either gitlab.com or github.com. Do NOT use public repository. All team members must contribute meaningfully to the repository:

- At least one **substantial** commit per team member per week (we will check this subjectively during the demo)
- At least one merge commit must be present in the final submission
- Commit messages must be descriptive and reflect the actual change made

8. Submission Checklist (in final gitlab/github repo)

8.1 Code

- `chat.py` runs from the command line with a vocabulary file path
- `analyze.py` runs from the command line with a chat file path
- `data.json` is generated correctly and matches what `app.js` expects
- The browser experience works on any machine with no hardcoded absolute paths
- Per-person profile slides are implemented and navigable

8.2 Documentation

- README explaining how to run the project and describing the data.json schema
- Inline comments in analyze.py for every non-obvious decision or definition

8.3 Report (LaTeX)

The report must include:

1. A list of external libraries used and their purpose
2. A description of all features implemented
3. An explanation of your custom statistic and why you chose it
4. The definition choices made in the parser (e.g. what counts as a reply, what is a silence), with justification
5. Hurdles faced and how you overcame them
6. What you would do differently with 10 more hours.
7. Bibliography with references and links to external sources
8. Write a suitable Makefile to compile the latex report

9. Marks Distribution (Total=18)

Category	Marks
Proper Git commits	2
Functionality of code	5
Report + Makefile	2
Code quality (modularity, inline comments)	2
Viva	4
Ingenuity	3

10. For those who wanted more (Optional - You won't get extra marks for this)

- I. Some of you wanted to make the sentences meaningful to add flavour to the project. For this and only this task you can use the nltk

library. Thus, you can make your chat.py such that for the given vocabulary.txt it gives meaningful sentences. Keep it such that on changing a boolean to false, it would remove all the hardcoding and the code would work on any vocabulary.txt . Thus, the import statement of nltk should also be called only if the boolean is true.

- II. Your chat.py can generate one more file named online.txt that keeps data of when the users were online. The exact details of how online.txt should look like is left for you.

11. Some clarifications

1. The sentences can include the names of the group chat members and the special symbol @ (Try to use this for getting better statistics). Thus allowed words = (Vocabulary.txt) U {@} U (Names of group chat members). Note that if the name has a last name then make sure that the last name is always used with the corresponding first name.
2. For plotting graphs, you can use “chart.js”. For the demo, you will be provided with chart.js, which you can copy into the desired location.
3. The messages are NOT mandated to be meaningful.
4. The length of the chat should be good enough to clearly differentiate the personalities.
5. While the chat generator script is required to clearly make some personalities different, make sure all this is done probabilistically. For example, a night owl has a higher probability to message at night but there should be some non-zero chance of a night owl’s message in the morning. (As in real life :)